GESP C++ 三級

練習題本

照片卷 40 題 + 2026 年 09、06、03 月三份試卷

本冊內容	使用方式
照片卷重排版	保留 1 至 40 題編號和選項順序；移除手寫答案。
三份試卷練習版	保留題目、圖表、輸入輸出與樣例；移除答案表與參考程式。
總題量	115 題客觀題 + 6 題編程題；重複題保留，方便按卷練習。
配套詳解	照片卷全部解析、三卷客觀題逐題解析、6 題編程解法與程式。

先獨立作答，再對照《完整詳解》。遇到不確定的題，留下計算過程，比只改答案更容易找出觀念漏洞。

版本說明：月份與題名依你提供的 PDF 標示。照片題目作繁體重排，部分語句為可讀性而整理。完整原始 PDF 保留在練習包內。

照片卷的手寫圈選不作答案依據。本冊按 ASCII、題目指定的位寬與 C++11 作答慣例解說；單位與措辭歧義見詳解冊。

作答與複習記錄

照片卷：選擇題填 A/B/C/D；判斷題填 T/F。第 1 至 14 題包含混合題型；第 15 至 26 題為選擇；第 27 至 40 題為判斷。

題號	答案	題號	答案	題號	答案	題號	答案
1	_____	2	_____	3	_____	4	_____
5	_____	6	_____	7	_____	8	_____
9	_____	10	_____	11	_____	12	_____
13	_____	14	_____	15	_____	16	_____
17	_____	18	_____	19	_____	20	_____
21	_____	22	_____	23	_____	24	_____
25	_____	26	_____	27	_____	28	_____
29	_____	30	_____	31	_____	32	_____
33	_____	34	_____	35	_____	36	_____
37	_____	38	_____	39	_____	40	_____

試卷	作答日期	客觀題 /50	編程題 /50	重做日期
2026 /09	_____	_____	_____	_____
2026 /06	_____	_____	_____	_____
2026 /03	_____	_____	_____	_____

建議練習節奏

第一次：完成照片卷，找出進制、補碼、位運算、字串與陣列的薄弱處。
第二次：依序完成三卷；每卷可自訂 90 分鐘，這是練習建議，不是官方時間聲明。
第三次：只重做錯題，並在不看解法的情況下獨立重寫 6 題程式。

照片卷｜第 1 至 40 題

重排自兩張「高二資訊科技考前三級真題專題練習」照片。未加入照片上的班級、姓名與手寫批註。

1 三進制數 2102 轉換成十進制是多少？

A. 63

B. 65

C. 67

D. 69

作答：_____ 把握度：高 / 中 / 低

---

2 8 位二進制下，十進制數 -15 的補碼是？

A. 11110000

B. 10001111

C. 10010000

D. 11110001

作答：_____ 把握度：高 / 中 / 低

---

3 關於計算機的二進制編碼表示，下列說法錯誤的是？

A. 原碼最高位為符號位，0 表正、1 表負，其餘位表示絕對值。

B. 正數補碼與原碼相同；求負數補碼，可將對應正數的位元全部取反再加 1。

C. 求負數補碼時，可從對應正數的最右邊保留到第一個 1，左側全部取反。

D. n 位補碼的表示範圍為 [-2^(n-1), +2^(n-1)]。

作答：_____ 把握度：高 / 中 / 低

---

4 判斷：求 -11 的補碼，可從 +11 的二進制最右端掃描，遇到第一個 1 後，將左邊各位全部取反。

作答：_____ 把握度：高 / 中 / 低

---

5 在 32 位 int、補碼表示的前提下，下列按位取反的說法錯誤的是？

A. ~5 的結果為 -6。

B. ~0 的結果為 0。

C. ~(-3) 的結果為 2。

D. ~8 的結果為 -9。

作答：_____ 把握度：高 / 中 / 低

---

6 判斷：若 int 變數 a 的二進制最低位為 0，則 ((a & 3 & 1) == 0) 為 true。

作答：_____ 把握度：高 / 中 / 低

---

7 若字元變數 _1 的值是字元 '1'，則 (int)_1 的值是？

A. 1

B. -1

C. 49

D. +1 或 -1

作答：_____ 把握度：高 / 中 / 低

---

8 關於 string 的成員函式，下列說法錯誤的是？

A. size() 回傳字串長度，一般不計結尾終止空字元。

B. length() 與 size() 功能相同。

C. empty()：非空回傳 true，空字串回傳 false。

D. s.append(s2, 0, 3) 取 s2 從下標 0 開始的 3 個字元。

作答：_____ 把握度：高 / 中 / 低

---

9 判斷：下列程式中 x 與 y 相等，因為 char 與 int 可以互換。

```
char x = '1';
int y = 1;
```

作答：_____ 把握度：高 / 中 / 低

---

10 以下陣列定義，符合 C++ 語法的是？

A. int [10] a;

B. int b['&'];

C. int c[*];

D. double d[10.0];

作答：_____ 把握度：高 / 中 / 低

---

11 判斷：下列程式可把原陣列中所有大於 0 的元素改成 0。

```
int a[] = {2, -5, 7, 0, -1};
int n = sizeof(a) / sizeof(a[0]);
for (int num : a) {
    if (num > 0) num = 0;
}
```

作答：_____ 把握度：高 / 中 / 低

---

12 以下問題中，最不適合用枚舉法解決的是？

A. 找出 1 至 100 間所有能被 7 整除的數。

B. 找出 100 至 200 間所有質數。

C. 計算 1+2+3+...+1000。

D. 找出各位數字和等於 10 的三位數。

作答：_____ 把握度：高 / 中 / 低

---

13 雞兔同籠：35 個頭、94 隻腳。以下枚舉邏輯最合理的是？

A. 枚舉雞 x=0..35，兔 y=35-x，再檢查 2*x+4*y==94。

B. 枚舉兔 y=0..94，雞 x=35-y，再檢查腳數。

C. 枚舉 x=0..100、y=0..100，再同時檢查頭與腳數。

D. 枚舉腳數 sum=0..94，檢查 sum==94。

作答：_____ 把握度：高 / 中 / 低

---

14 判斷：模擬算法必須逐行複刻手工運算，途中不能加入任何條件判斷。

作答：_____ 把握度：高 / 中 / 低

---

15 10110101 是某數的 8 位補碼，該數的十進制值是？

A. -73

B. -75

C. -77

D. 75

作答：_____ 把握度：高 / 中 / 低

---

16 把相鄰兩個低 4 位的值合併為 1 個位元組。0x1、0x2、0x3、0x4 壓縮後是？

A. 12D、34D

B. 12Q、34Q

C. 12H、34H

D. 00010011B、00110101B

作答：_____ 把握度：高 / 中 / 低

---

17 unsigned char c=0x0F；執行 c=c<<3 後，c 的十進制值為？

A. 64

B. 72

C. 80

D. 120

作答：_____ 把握度：高 / 中 / 低

---

18 對 32 位有符號 int n，下列關於 ~ 的說法正確的是？

A. ~6 為 5。

B. 按位取反滿足 ~n 的數學值為 -n-1。

C. ~0 為 1。

D. ~(-2) 為 -1。

作答：_____ 把握度：高 / 中 / 低

---

19 計算 3 & 7 | 15 ^ ~11 的結果。

A. -9

B. -1

C. -7

D. -5

作答：_____ 把握度：高 / 中 / 低

---

20 執行下列程式後，sub 的值是？

```
string str = "GESP2026";
string sub = str.substr(4, 2);
```

A. 20

B. 02

C. 2026

D. 026

作答：_____ 把握度：高 / 中 / 低

---

21 關於 string 的成員函式，下列說法錯誤的是？

A. size() 與 length() 都回傳字串長度。

B. empty()：空字串為 true，非空為 false。

C. substr(pos) 省略長度時，從 pos 取到末尾。

D. s.erase(2) 只刪除下標 2 的一個字元。

作答：_____ 把握度：高 / 中 / 低

---

22 關於計算機的數據編碼，下列說法正確的是？

A. 10001101B 若按原碼解讀是 -13，若按無符號數解讀是 141。

B. 數據最終以二進制、八進制、十六進制三種形式存儲運算。

C. Byte 是最小數據單位，bit 是最小存儲單位。

D. 1k 位元組在本卷慣例下表示 1000 位元組。

作答：_____ 把握度：高 / 中 / 低

---

23 關於進制轉換，下列說法不正確的是？

A. 1101B = 1×2^3+1×2^2+0×2^1+1×2^0 = 13。

B. 42Q = 4×8+2 = 34。

C. 2BEH = 2×16^2+11×16+14 = 702。

D. 345Q = 0E3H（通常寫作 E3H）。

作答：_____ 把握度：高 / 中 / 低

---

24 按本卷 1 KB=1024 B 的慣例，2 KB 等於多少 bit？

A. 16384

B. 20000

C. 2000

D. 2048

作答：_____ 把握度：高 / 中 / 低

---

25 下列變數皆為正整數，哪個位運算說法錯誤？

A. num >> 1 等於 num 整數除以 2，向下取整。

B. num | 0x0F 可把低 4 位全設為 1，高位不變。

C. num ^ num 一定等於 0。

D. 右移丟棄低位後，結果一定等於數學上的精確除法。

作答：_____ 把握度：高 / 中 / 低

---

26 以下陣列定義，符合 C++ 語法的是？

A. int a[] = {1, 2, 3.0};

B. int b['A'];

C. int c[];

D. float d[10.0];

作答：_____ 把握度：高 / 中 / 低

---

27 判斷：反碼運算產生最高位進位時，需把進位加回結果最低位。

作答：_____ 把握度：高 / 中 / 低

---

28 判斷：8 位補碼表示範圍為 -127 至 +127。

作答：_____ 把握度：高 / 中 / 低

---

29 判斷：補碼加法最高位有進位時，需要把它循環加回結果最低位。

作答：_____ 把握度：高 / 中 / 低

---

30 判斷：8 位二進制原碼的有符號數範圍為 -128 至 +127。

作答：_____ 把握度：高 / 中 / 低

---

31 判斷：若 int a 滿足 ((a & 1) == 1)，則 a 是奇數。

作答：_____ 把握度：高 / 中 / 低

---

32 判斷：C++ 的位運算符有不同優先級，使用括號能明確指定分組。

作答：_____ 把握度：高 / 中 / 低

---

33 判斷：下列程式若能正確執行，會輸出 GESP。

```
int main() {
    string name = "GESP";
    cout << name[false] << name[true]
         << name[1 << 1] << name[7 >> 1] << endl;
}
```

作答：_____ 把握度：高 / 中 / 低

---

34 判斷：把下列包含行首數字的所有文字輸入編譯器，可正常編譯執行。

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 int main() {
4     cout << "Hello world!" << endl;
5     return 0;
6 }
```

作答：_____ 把握度：高 / 中 / 低

---

35 判斷：檢查 int 數字 s[i] 是否在 0 至 9，可寫 s[i]>='0' && s[i]<='9'。

作答：_____ 把握度：高 / 中 / 低

---

36 判斷：枚舉可以在保證不漏解的前提下縮小範圍，不必遍歷所有想像得到的值。

作答：_____ 把握度：高 / 中 / 低

---

37 判斷：((0xf0 + 0x15) == 255) 的值為 true。

作答：_____ 把握度：高 / 中 / 低

---

38 判斷：已知 'A' 的 ASCII 為 0x41，則 'm' 的 ASCII 八進制為 155。

作答：_____ 把握度：高 / 中 / 低

---

39 判斷：int arr[]={1,2,3}；若 int 佔 4 Byte，sizeof(arr) 為 12。

作答：_____ 把握度：高 / 中 / 低

---

40 判斷：下列程式會輸出 98。

```
#include <iostream>
using namespace std;
int main() {
    int a = 'a', b = 'b';
    if (a > 0 || b++ > 0) {
    }
    cout << b;
}
```

作答：_____ 把握度：高 / 中 / 低

2026 / 09 | 試卷練習

選擇題 15 題 × 2 分；判斷題 10 題 × 2 分；編程題 2 題 × 25 分。

以下保留所提供 PDF 的題目與原版頁碼。答案列與參考程式已移除，可在留白處寫思路；原題的樣例輸出仍完整保留。

選擇題作答

題號	答案	題號	答案	題號	答案	題號	答案
1	_____	2	_____	3	_____	4	_____
5	_____	6	_____	7	_____	8	_____
9	_____	10	_____	11	_____	12	_____
13	_____	14	_____	15	_____		

判斷題作答

題號	答案	題號	答案	題號	答案	題號	答案
1	_____	2	_____	3	_____	4	_____
5	_____	6	_____	7	_____	8	_____
9	_____	10	_____				

編程題檢查

1. 我能用一句話說清楚算法。
2. 我已手算樣例，確認輸入、輸出和下標。
3. 我已測試最小值、邊界、重複值與特殊字元。
4. 我能說明時間複雜度，並指出至少一種常見錯法。

CCF 编程能力等级认证
Grade Examination of Software Programming

C++ 三级

2026 年 09 月

1 单选题（每题 2 分，共 30 分）

第 1 题 关于计算机的数据编码，下面说法不正确的是（ ）。

- A. 数字 5，不管采用原码、反码、补码，它的 8 位二进制形式都是 $(0000\ 0101)_2$。
- B. 计算机科学中，单位换算默认 1 KB = 1024 B，而非 1000 B。
- C. 原码是最直观的一种有符号数表示方法。
- D. 反码运算中，最高位进位要抛掉。

第 2 题 关于进制转换，下面说法不正确的是（ ）

- A. 数字 1101B 可以分解为：

  $1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 8 + 4 + 0 + 1 = 13$（十进制）
- B. 数字 42Q 可以分解为：

  $4 \times 8^1 + 2 \times 8^0 = 32 + 2 = 34$（十进制）
- C. 数字 2BEH 可以分解为：

  $2 \times 16^2 + 11\,(\mathrm{B}) \times 16^1 + 14\,(\mathrm{E}) \times 16^0 = 512 + 176 + 14 = 702$（十进制）
- D. 345Q = 0E3H（通常写作 E3H）

第 3 题 下面程序的运行结果是（ ）

```cpp
#include <iostream>
using namespace std;
int main() {
    int x = 1;
    for (int i = 0; i = 0; i <= 5) {
        x++;
    }
    cout << x << endl;
    return 0;
}
```

- A. `1`
- B. `10`
- C. `15`
- D. 程序编译错误

第 4 题 关于计算机编码中补码，下面说法错误的是（　　）。

- [ ] **A.** 正数的补码：与其原码、反码相同。
- [ ] **B.** 负数的补码：将其对应的正数的原码按位取反（得到反码），然后加 1。
- [ ] **C.** 8 位的补码，最大表示数据是 128，最小表示数据是 −128。
- [ ] **D.** 负数的补码，从右往左扫描正数的二进制形式，遇到第一个 1 之后，左边的所有位都取反。

第 5 题 下面程序执行的结果是（　　）。

```cpp
#include <iostream>
using namespace std;
int main() {
    int x = 2026;
    int count = 0;
    while (x > 0) {
        count++;
        x = x & (x - 1);
    }
    cout << count << endl;
}
```

- [ ] **A.** `2026`
- [ ] **B.** `8`
- [ ] **C.** `16`
- [ ] **D.** `32`

第 6 题 关于位运算，下列说法错误的是（　　）。

- [ ] **A.** 找唯一数：数组中唯一出现一次的数，其余出现两次，全部异或结果即为该数。
  例子：数组 `[5, 7, 9, 7, 5]`（唯一数是 `9`）
- [ ] **B.** 若左移导致高位溢出（如超过整型范围），结果不再符合乘法规律。
- [ ] **C.** 有符号整数左移溢出时结果确定。
- [ ] **D.** $n+2$ 位二进制数 $A_1A_2\cdots A_n00$，一定是 4 的倍数（最后两位 00 是二进制的 00）。

第 7 题 计算 `3 & 7 | 15 ^ ~11`（`a=3, b=7, c=15, d=11`）：（　　）。

- [ ] **A.** `-9`
- [ ] **B.** `-1`
- [ ] **C.** `-7`
- [ ] **D.** `-5`

第 8 题 下面函数是一个加密解密的函数，横线处应该填入的程序段是（　　）。

```cpp
#include <iostream>
#include <string>
using namespace std;
int main() {
    int key;
    cin >> key;
    string msg = "Hello GESP";
    cout << msg << endl;
    for (int i = 0; i < msg.size(); i++) {
        msg[i] = msg[i] ^ key;
    }
    cout << msg << endl;
    ---------------------------------
    cout << msg << endl;
    return 0;
}
```

☐ **A.**

```cpp
for (int i = 0; i < msg.size(); i++) {
    msg[i] = msg[i] | key;
}
```

☐ **B.**

```cpp
for (int i = 0; i < msg.size(); i++) {
    msg[i] = msg[i] >> key;
}
```

☐ **C.**

```cpp
for (int i = 0; i < msg.size(); i++) {
    msg[i] = msg[i] ^ key;
}
```

☐ **D.**

```cpp
for (int i = 0; i < msg.size(); i++) {
    msg[i] = msg[i] & key;
}
```

第 9 题 下列程序是分别判断输入的 `N` 个数是否可能是二进制、八进制、十进制、十六进制，横线处应该填入的是（　　）

```cpp
#include <iostream>
using namespace std;
int main() {
    int n = 0;
    cin >> n;
    for (int i = 0; i < n; i++) {
        char str[11];
        cin >> str;
        char max = '0';
        for (int i = 0; str[i] != '\0'; i++)
            if (str[i] > max)
                max = str[i];
        ----------------------------
    }
    return 0;
}
```

- [ ] **A.** `cout << (max <= '1') << " " << (max <= '7') << " " << (max <= '9') << " " << (max <= 'F') << endl;`
- [ ] **B.** `cout << (max <= 1) << " " << (max <= 7) << " " << (max <= 9) << " " << (max <= 'F') << endl;`
- [ ] **C.** `cout << (max <= 1) << " " << (max <= 7) << " " << (max <= 9) << " " << (max <= F) << endl;`
- [ ] **D.** `cout << (max >= 1) << " " << (max >= 7) << " " << (max >= 9) << " " << (max >= F) << endl;`

第 10 题 下面的程序是位掩码技术实现权限的组合管理。首先定义四个权限常量，分别对应读、写、修改、删除，每个权限占用整数的一个独立二进制位。初始化权限掩码 `mask` 为 `0`，代表没有任何权限。如果想实现如下的操作：添加读权限；添加写权限；清除读权限，其他位不变；添加删除权限。则横线处应该填入的是（　　）。

```
#include <iostream>
using namespace std;

int main() {
    const int READ = 1 << 0;
    const int WRITE = 1 << 1;
    const int MODIFY = 1 << 2;
    const int DELETE_ = 1 << 3;
    int mask = 0;
    ________________________
    return 0;
}
```

- [ ] **A.**

```
mask = mask | READ;
mask = mask | WRITE;
mask = mask & (~READ);
mask = mask | DELETE_;
```

- [ ] **B.**

```
mask = mask & READ;
mask = mask & WRITE;
mask = mask | (~READ);
mask = mask & DELETE_;
```

- [ ] **C.**

```
mask = mask | READ;
mask = mask & WRITE;
mask = mask & (~READ);
mask = mask | DELETE_;
```

- [ ] **D.**

```
mask = mask | READ;
mask = mask | WRITE;
mask = mask | (~READ);
mask = mask | DELETE_;
```

第 11 题 下面选项中提到的变量都是正整数，关于位运算，下面说法错误的是（　　）。

- [ ] **A.** `num >> 1`，对于正整数，等价于对 `num` 做整数除以 2，向下取整
- [ ] **B.** `num | 0x0F`，可以把 `num` 的低 4 位全部置为 `1`，高位保持不变

- [ ] **C.** `num ^ num` 的结果一定等于 `0`
- [ ] **D.** 若 `num` 右移时低位被移出丢弃，结果一定符合除法数学规律

第 12 题 已知：`a=0, b=2, c=11, d=7, e=5`，表达式 `!a & b << c ^ d | e` 的结果是（　　）

- [ ] **A.** `5`
- [ ] **B.** `13`
- [ ] **C.** `7`
- [ ] **D.** `28`

第 13 题 关于 `string` 的成员函数，下面说法错误的是（　　）

- [ ] **A.** `size()` 返回字符串的字符个数，不包含末尾的 `'\0'`；`length()` 和 `size()` 功能完全等价
- [ ] **B.** `s.empty()`：字符串为空时返回 `true`，字符串不为空返回 `false`
- [ ] **C.** `s.substr(pos, len)`，当省略第二个参数 `len` 时，代表从 `pos` 位置一直截取到字符串末尾
- [ ] **D.** `s.erase(2)` 表示删除字符串中下标为 2 的这一个字符

第 14 题 以下数组定义，符合 C++ 语法的是（　　）

- [ ] **A.** `int a[] = {1, 2, 3.0};`
- [ ] **B.** `int b['A'];`
- [ ] **C.** `int c[ ];`
- [ ] **D.** `float d[10.0];`

第 15 题 三进制有个奇妙的应用，有 1000 个酒缸，恰好 1 缸有毒。毒酒喝下后 24 小时毒发死亡；无毒酒无影响。总共有 48 小时，可以做两轮实验，每轮可以重新安排老鼠喝酒。有三种情况，这只老鼠两轮都不喝这缸酒；这只老鼠第一轮喝这缸酒；这只老鼠第二轮喝这缸酒，要保证找出毒酒，最少需要多少只老鼠？（　　）

- [ ] **A.** 5
- [ ] **B.** 7
- [ ] **C.** 10
- [ ] **D.** 9

2　判断题（每题 2 分，共 20 分）

第 1 题 对于计算机编码运算，补码做加法运算时，如果最高位产生进位，需要把该进位循环加到结果的最低位。

第 2 题 求 -13 的补码，可以对正数 13 的二进制，从右向左扫描，遇到第一个 1 之后，右边所有位全部取反，左边保持不变。

第 3 题 一个 8 位二进制原码表示的有符号数，可表示的数值范围是 -128 到 $+127$。

第 4 题 这段代码执行后，能够将数组中所有大于 `0` 的元素赋值为 `0`。

```
int a[] = {2, -5, 7, 0, -1};
int n = sizeof(a) / sizeof(a[0]);
for (int num : a) {
    if (num > 0) {
        num = 0;
    }
}
```

第 5 题 `a` 是 `int` 有符号整型，若 `(a & 1) == 1` 结果为 `true`，则 `a` 一定是正奇数。

第 6 题 `char str[4] = "GESP";` 将在内存空间中给 `str[4]` 分配 `5` 个字节。

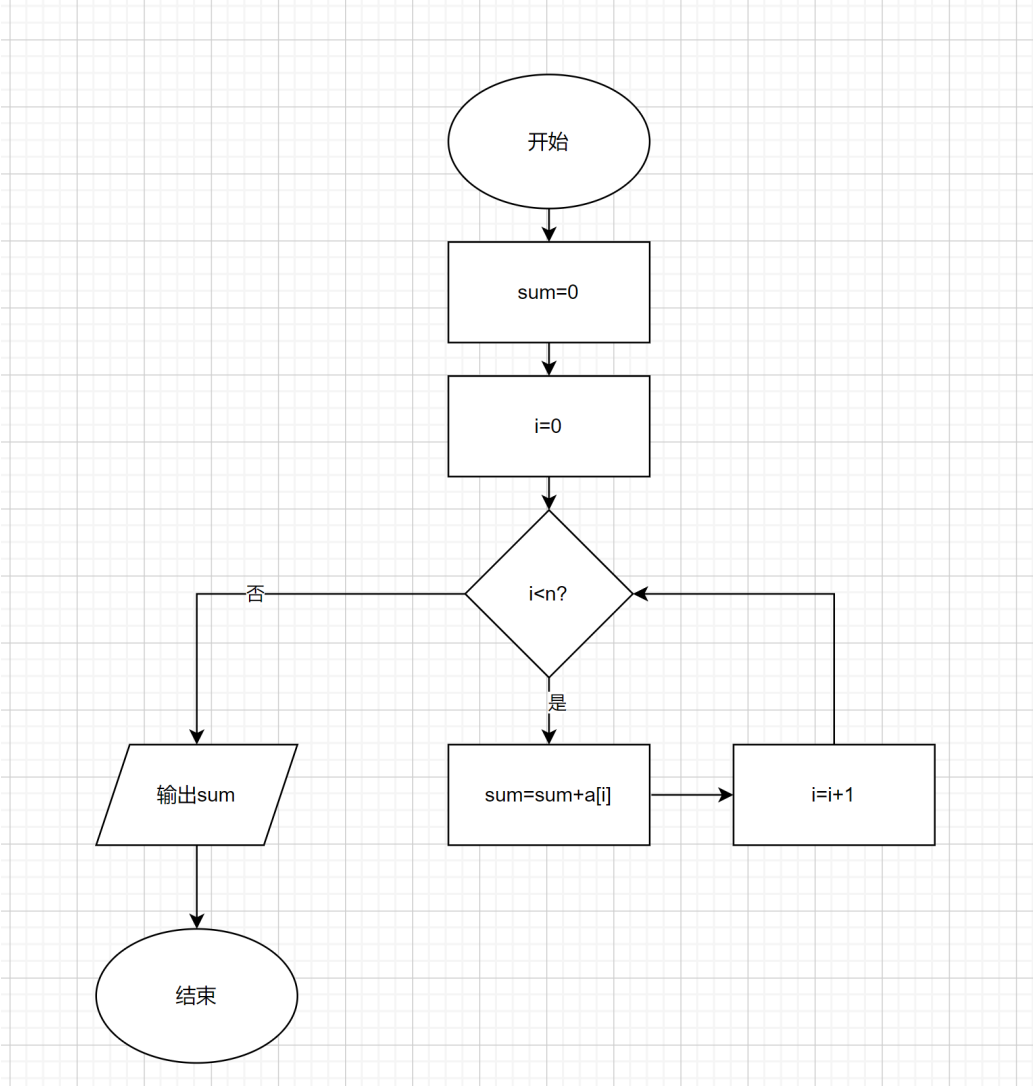
第 7 题 判断字符串中字符是否是 `'0'` 或者 `'1'`，可以写条件：`if(str[i]==0 || str[i]==1)`。

第 8 题 下面程序执行后，将输出 `98`。

```
#include <iostream>
using namespace std;
int main() {
    int a = 'a', b = 'b';
    if (a > 0 || b++ > 0) {
    }
    cout << b;
}
```

第 9 题 枚举可以在保证不漏解的前提下缩小枚举范围，不是必须遍历全部。

第 10 题 下面流程图逻辑：初始化 $\text{sum} \leftarrow 0$，$i \leftarrow 0$；当 $i < n$ 成立时，执行 $\text{sum} \leftarrow \text{sum} + a[i]$，再执行 $i \leftarrow i + 1$；循环结束输出 `sum`。该流程图可以正确计算一维数组 `a[0]` ~ `a[n]` 全部元素的总和。



3 编程题（每题 25 分，共 50 分）

3.1 编程题 1

- **试题名称**：公共二进制位
- **时间限制**：1.0 s
- **内存限制**：512.0 MB

3.1.1 题目描述

小红有 n 个非负整数 $a_1, a_2, \cdots, a_n$。她将每个整数转换为二进制后，想知道有多少个二进制位在所有整数中均为 1。

二进制位从右向左编号为 $0, 1, 2, \cdots$。若某个整数的二进制表示中没有第 k 位，则认为它的第 k 位为 0。

请你求出满足条件的二进制位数量。

3.1.2 输入格式

- 第一行一个整数 n，表示整数的个数。
- 第二行 n 个非负整数 $a_1, a_2, \cdots, a_n$。

3.1.3 输出格式

输出一个整数，表示所有整数的二进制表示中均为 1 的二进制位数量。

3.1.4 样例

3.1.5 输入样例 1

```
3
13 7 15
```

3.1.6 输出样例 1

```
2
```

3.1.7 样例解释 1

三个整数的二进制表示分别为 $(1101)_2$、$(0111)_2$ 和 $(1111)_2$。

其中第 0 位和第 2 位均为 1，因此答案为 2。

3.1.8 数据范围

$2 \leq n \leq 20000$，$0 \leq a_i \leq 10^9$。

3.2 编程题 2

- **试题名称**：分割字符串
- **时间限制**：1.0 s
- **内存限制**：512.0 MB

3.2.1 题目描述

小明拿到了一段字符串文本，小明希望用下面的规则分割这个字符串：

1. 小明从左到右扫描字符串，遇到**第一个空格**后停止，并将字符串分割为左右两个部分（舍弃用于分割的这个空格）；
2. 对于上次分割后右侧的字符串，小明继续扫描，第一次遇到**连续两个空格**后停止，并将字符串分割为左右两个部分（舍弃用于分割的这两个空格）；
3. 对于上次分割后右侧的字符串，小明继续扫描，第一次遇到**连续三个空格**后停止，并将字符串分割为左右两个部分（舍弃用于分割的这三个空格）；
4. 以此类推，小明每分割一次，下一次分割就需要更长的连续空格；直到字符串无法再次分割。

最后输出每次分割的左侧的字符串，以及最后一次分割的右侧字符串。
如果字符串完全无法进行任何一次分割，直接输出其本身。

3.2.2 输入格式

输入一行，一个字符串。

3.2.3 输出格式

输出若干行。每行为分割后的一个字符串，注意依次输出：
第一次分割左侧的字符串，第二次分割左侧的字符串，...，最后一次分割左侧的字符串，最后一次分割右侧的字符串。

3.2.4 样例

3.2.5 输入样例 1

```
sdfd 55 55   99312
```

3.2.6 输出样例 1

```
sdfd
55 55
 99312
```

3.2.7 样例解释 1

1. 第一次分割将字符串分割为 `sdfd` 和 `55 55   99312`；
2. 第二次分割将字符串分割为 `55 55` 和 ` 99312`。

对于第二次分割，注意 `55` 和 `55` 之间虽然有空格，但是第二次分割需要连续两个空格才行；

另外，`55 55` 和 `99312` 之间有三个空格，第二次分割只需要两个，所以会分割为 `55 55` 和 ` 99312`，也就是右侧会留下一个空格；因为只留下了一个空格，所以不再满足第三次分割要求。

3.2.8 输入样例 2

```
AbCd   999
```

3.2.9 输出样例 2

```
AbCd

999
```

3.2.10 样例解释 2

1. 第一次分割将字符串分割为 `AbCd` 和 `  999`；
2. 第二次分割将字符串分割为 `空字符串` 和 `999`。

由于 `AbCd` 和 `999` 之间有三个空格，第一次分割消耗掉一个，因此还能进行第二次分割，这个时候会分割出空字符串，因此输出中会输出一个空行（不能省略）。

3.2.11 数据范围

字符串长度不超过 1000，字符串中只包含大写字母、小写字母，数字，**或空格**，且输入字符串不会以空格结尾。

特别强调：输入仅一行，但可能包含**若干空格**

2026 / 06 | 試卷練習

選擇題 15 題 × 2 分；判斷題 10 題 × 2 分；編程題 2 題 × 25 分。

以下保留所提供 PDF 的題目與原版頁碼。答案列與參考程式已移除，可在留白處寫思路；原題的樣例輸出仍完整保留。

選擇題作答

題號	答案	題號	答案	題號	答案	題號	答案
1	_____	2	_____	3	_____	4	_____
5	_____	6	_____	7	_____	8	_____
9	_____	10	_____	11	_____	12	_____
13	_____	14	_____	15	_____		

判斷題作答

題號	答案	題號	答案	題號	答案	題號	答案
1	_____	2	_____	3	_____	4	_____
5	_____	6	_____	7	_____	8	_____
9	_____	10	_____				

編程題檢查

1. 我能用一句話說清楚算法。
2. 我已手算樣例，確認輸入、輸出和下標。
3. 我已測試最小值、邊界、重複值與特殊字元。
4. 我能說明時間複雜度，並指出至少一種常見錯法。

CCF 编程能力等级认证
Grade Examination of Software Programming

C++ 三级

2026 年 06 月

1 单选题（每题 2 分，共 30 分）

第 1 题 关于计算机的数据编码，下面说法正确的是（ ）。

- A. 机器数 `1000 1101B` 的真值可能是 `-13`（如果它是原码）或 `141`（如果它是无符号数）等数，这取决于我们如何解释它。
- B. 计算机中，所有数据最终都以二进制、八进制、十六进制的形式存储和运算。
- C. 字节（`Byte`）是计算机中最小的数据单位。位（`bit`）是计算机中最小的存储单位。
- D. 计算机中，`1k` 字节是 `1000` 字节的意思。

第 2 题 计算机厂商为了计算方便，一般采用 `1000` 进制。如果我们买的厂商标注的是 `1 TB` 的硬盘，它实际的存储容量是（ ）。

- A. $1000 \times 1000 \times 1000 \times 1000 \div 1024 \div 1024 \div 1024\text{b} = 931\text{Gb}$
- B. $1000 \times 1000 \times 1000 \times 1000 \div 1024 \div 1024 \div 1024\text{B} = 931\text{GB}$
- C. $1024 \times 1024 \times 1024 \times 1024 \div 1000 \div 1000 \div 1024\text{B} = 1049\text{GB}$
- D. $1000 \times 1024 \times 1024 \times 1024 \div 1024 \div 1024 \div 1024\text{b} = 977\text{Gb}$

第 3 题 低 4 位、高 4 位压缩技术，适用于数据仅使用字节的一部分（如仅用低 4 位）的场景。字节结构：一个字节为 8 位，分为高 4 位（高位）和低 4 位（低位）。当数据是十六进制数（$0 \sim 15$，即 `0x0` 到 `0xF`），每个值仅需 4 位表示，高 4 位全为 `0`。将两个相邻的 4 位值合并为一个字节。四个数据 `0x1`、`0x2`、`0x3`、`0x4` 采用上述压缩技术压缩以后是（ ）。

- A. `12D`、`34D`
- B. `12Q`、`34Q`
- C. `12H`、`34H`
- D. `00010011B`、`00110101B`

第 4 题 关于计算机编码中反码和补码，下面说法错误的是（ ）。

- A. 负数的补码，一个快速方法是从右往左扫描正数的二进制形式，遇到第一个 `1` 之后，左边的所有位都取反。
- B. 对于一个 n 位的二进制数：最大表示范围：$[-(2^{n-1}) - 1, +(2^{n-1} - 1)]$。
- C. 反码减法可以统一为加法。符号位可以直接参与运算。
- D. 反码表示中，0 的表示不唯一：`0000 0000B` 和 `1111 1111B`。

第 5 题 一种加密方式是字符数组与密钥 `KEY`、运算方式分开传输，比如字符数组 `char text[4] = {'G', 'E', 'S', 'P'};` 由一种传输方式发送，密钥 `KEY = 2026` 通过另一种发送方式发送，运算方式 `char function[4] = {'|', '-', '^', '+'};` 又是另一种发送方式发送。三种数据都到达目的地以后，分别进行例如 `'G' | 6`、`'E' - 2`、`'S' ^ 0`、`'P' + 2` 等计算，来得到相应的真实内容，上述 `GESP` 通过这种加密方式，加密以后最终的内容是（ ）。

- A. `GCSR`
- B. `RSCG`
- C. `GCSA`
- D. `BCSR`

第 6 题 关于位运算，下列说法错误的是（ ）。

- A. 找唯一数：数组中唯一出现一次的数，其余出现两次，全部异或结果即为该数。例如：数组 `[5, 7, 9, 7, 5]`（唯一数是 `9`）。
- B. 交换两个数：`a ^= b; b ^= a; a ^= b;`（无需临时变量）。
- C. 将二进制位整体左移 n 位，高位溢出舍弃，低位补 `0`；等价于 `num` 乘以 2^n。
- D. 对每一个二进制位取反，包括符号位，简单运算规则是 `~n = -n - 1`。

第 7 题 关于字符串和字符数组，下列说法正确的是（ ）。

- A.

```
char str[] = "GESP";
int len1 = sizeof(str);
int len2 = strlen(str);
```

上面程序能够正确执行，`len1` 与 `len2` 相等。

- B.

```
char str1[4] = "GESP";
char str2[4] = {'G', 'E', 'S', 'P'};
```

这段程序将能够正确执行。

- C.

```
char str2[4] = {'G', 'E', 'S', 'P'};
strcpy(str2, "HELLO,GESP");
cout << str2 << endl;
```

这段程序即使能够运行，但是存在覆盖数组以外的内存空间的行为，可能会引起严重错误。

- D.

```
char dest[4] = {'G', 'E', 'S', 'P'};
char src[] = "HELLO";
strcat(dest, src);
cout << dest << endl;
```

这段程序能够正确执行，不存在数组越界行为。

第 8 题 计算机中的 `2 KB` 等于多少 `bit`（ ）。

- A. `16384`
- B. `20000`

- [ ] **C.** `2000`
- [ ] **D.** `2048`

第 9 题 在 C++ 中，对于 32 位有符号整数 `int` 类型数据 `n`，关于按位取反运算符 `~`，下列说法正确的是（ ）。

- [ ] **A.** `~6` 的结果是 `5`。
- [ ] **B.** 按位取反满足公式 `~n = -n - 1`。
- [ ] **C.** `~0` 的结果是 `1`。
- [ ] **D.** `~(-2)` 的结果是 `-1`。

第 10 题 关于计算机中的二进制编码表示，下列说法错误的是（ ）。

- [ ] **A.** 原码是最直观的一种有符号数表示方法。最高位（最左边的位）为符号位：`0` 表示正数，`1` 表示负数，其余位为数值位（真值的绝对值）。
- [ ] **B.** 补码完美解决了原码和反码的缺陷，是现代计算机中表示有符号整数的标准方式。正数的补码与其原码、反码相同；负数的补码是将其对应正数的原码按位取反（得到反码），然后加 `1`。
- [ ] **C.** 计算补码的一个更快的技巧：从右往左扫描正数的二进制形式，遇到第一个 `1` 之后，左边的所有位都取反。
- [ ] **D.** 对于一个 n 位的二进制数，补码最大表示范围为 $[-2^{n-1}, +2^{n-1}]$。

第 11 题 下面选项中提到的变量都是正整数，关于位运算，下面说法错误的是（ ）。

- [ ] **A.** `num & 1`，结果为 `1` 则奇数，`0` 则偶数（仅看最低位）。
- [ ] **B.** `num & 0xFF` 保留低 8 位。
- [ ] **C.** `num & b` 的结果一定小于等于 `num`。
- [ ] **D.** 若 `num` 左移导致高位溢出（如超过整型范围），结果符合乘法规律。

第 12 题 `a=7, b=3, c=14, d=15, e=8`，对于运算表达式 `!a << b & c ^ d | e` 的结果是（ ）。

- [ ] **A.** `0`
- [ ] **B.** `15`
- [ ] **C.** `7`
- [ ] **D.** `14`

第 13 题 关于 `string` 的成员函数，下面说法错误的是（ ）。

- [ ] **A.** `size()`：返回字符串长度（字符个数，不含 `'\0'`）。
- [ ] **B.** `length()` 与 `size()` 功能完全一致，返回字符串长度。
- [ ] **C.** `empty()`：判断字符串是否为空（非空返回 `true`，空返回 `false`）。
- [ ] **D.** `s.append(s2, 0, 3);` 从 `s2` 下标 `0` 开始，截取 `3` 个字符。

第 14 题 以下数组定义，符合 C++ 语法的是（ ）。

- [ ] **A.** `int [10] a;`
- [ ] **B.** `int b['&'];`
- [ ] **C.** `int c[*];`
- [ ] **D.** `double d[10.0];`

第 15 题 现在有一个数，请你分别判断它们是否可能是二进制、八进制、十进制、十六进制。例如，`6AFF` 就只可能是十六进制，而 `1011` 则是四种进制皆有可能。输入 `N`（保证 $1 \leq N \leq 1000$），表示有 `N` 个数让你进行判断，接下来输入 `N` 个字符串（保证所有字符串长度不超过 `10`），判断可能是四个进制当中的哪个进制数。输出 `N` 行，每行 `4` 个数，用空格隔开，分别表示给定的字符串是否可能表示一个二进制数、八进制数、十进制数、十六进制数。使用 `1` 表示可能，使用 `0` 表示不可能。下面程序横线处可以满足这个要求的是（ ）。

```
#include <iostream>
using namespace std;
int main() {
    int n = 0;
    cin >> n;
    for (int i = 0; i < n; i++) {
        char str[11];
        cin >> str;
        char max = '0';
        for (int i = 0; str[i] != '\0'; i++)
            if (str[i] > max)
                max = str[i];
        ----------------------------------------
    }
    return 0;
}
```

- A. `cout << (max >= '1') << " " << (max >= '7') << " " << (max >= '9') << " " << (max >= 'F') << endl;`
- B. `cout << (max <= '1') << " " << (max <= '7') << " " << (max <= '9') << " " << (max <= 'F') << endl;`
- C. `cout << (max = '1') << " " << (max = '7') << " " << (max = '9') << " " << (max = 'F') << endl;`
- D. `cout << (max < '1') << " " << (max < '7') << " " << (max < '9') << " " << (max < 'F') << endl;`

2 判断题（每题 2 分，共 20 分）

第 1 题 反码运算时，产生的进位需要循环进位，即最高位产生的进位要加回到结果的最低位。

第 2 题 `-11` 的补码的一种计算方式是从右往左扫描正数的二进制形式，遇到第一个 `1` 之后，左边的所有位都取反。

第 3 题 一个 8 位的二进制数补码，最大的表示范围是从 -127 到 $+127$。

第 4 题 判断某个数是否是质数，枚举范围可适当缩小（遍历到 $\sqrt{i}$ 而非 i），提升效率。

第 5 题 如果 `a` 为 `int` 类型的变量，且表达式 `((a & 1) == 1)` 的值为 `true`，则说明 `a` 是奇数。

第 6 题 十六进制数 `CCF` 对应的二进制数、八进制数、十进制数分别是：`110011001111`、`6317`、`3269`。

第 7 题 下列程序如果能够正确执行，那么输出的结果是 `GESP`。

```cpp
int main() {
    string name = "GESP";
    cout << name[false] << name[true] << name[1 << 1] << name[7 >> 1] << endl;
}
```

第 8 题 某个初学 C++ 的学生，在对照参考程序写了一个程序以后，信心满满地进行编译，他敲进编译器的全部代码如下，这个程序能够正常编译运行。

```cpp
1 #include<bits/stdc++.h>
2 using namespace std;
3 int main() {
4     cout << "Hello world!" << endl;
5     return 0;
6 }
```

第 9 题 代码逐行复刻手工运算步骤，中间不能增加任何判断逻辑，否则就不属于模拟算法。

第 10 题 判断一个 `int` 型数字 `s[i]` 是不是在 `0` 和 `9` 之间（可以是 `0` 和 `9`），判断条件可以写为 `if (s[i] >= '0' && s[i] <= '9')`。

3 编程题（每题 25 分，共 50 分）

3.1 编程题 1

- **试题名称**：加密
- **时间限制**：1.0 s
- **内存限制**：512.0 MB

3.1.1 题目描述

小杨同学有一串数字，想把它们变成另一串数字，这个过程叫做**加密**。

他有一本**密码本**，密码本告诉你：
每个数字应该变成哪个数字。

数字一共有 10 个：
`0`、`1`、`2`、`3`、`4`、`5`、`6`、`7`、`8`、`9`

密码本会依次告诉你：
`0` 要变成什么
`1` 要变成什么
`2` 要变成什么
……
`9` 要变成什么

请你按照密码本，把原来的每个数字都换成新的数字，然后输出。

3.1.2 输入格式

输入共有 3 行。
第一行：一个整数，表示有多少个数字需要加密；
第二行：这些需要加密的数字；
第三行：密码本，一共 10 个数字。

这 10 个数字的意思是：
第 1 个数字：表示 `0` 加密后变成什么；
第 2 个数字：表示 `1` 加密后变成什么；
第 3 个数字：表示 `2` 加密后变成什么；
……
第 10 个数字：表示 `9` 加密后变成什么。

3.1.3 输出格式

输出加密后的数字。

也就是：把输入第二行里的每个数字，都按照输入第三行的密码本换掉后输出。

3.1.4 样例

3.1.5 输入样例 1

```
7
0 2 0 3 4 1 9
9 0 1 2 3 4 5 6 7 8
```

3.1.6 输出样例 1

```
9 1 9 2 3 0 8
```

3.1.7 样例解释

第二行要加密的数字是：
`0 2 0 3 4 1 9`

第三行密码本是：
`9 0 1 2 3 4 5 6 7 8`

它的意思是：
`0` 变成 `9`
`1` 变成 `0`
`2` 变成 `1`
`3` 变成 `2`
`4` 变成 `3`
`5` 变成 `4`
`6` 变成 `5`
`7` 变成 `6`
`8` 变成 `7`
`9` 变成 `8`

所以：
`0` 变成 `9`
`2` 变成 `1`
`0` 变成 `9`
`3` 变成 `2`

`4` 变成 `3`
`1` 变成 `0`
`9` 变成 `8`

最后得到：
`9 1 9 2 3 0 8`

3.1.8 数据范围

需要加密的数字个数不超过 20000 个，且均为 0 到 9；密码本中的数字不重复，且均为 0 到 9。

3.2 编程题 2

- **试题名称**：字符转换
- **时间限制**：1.0 s
- **内存限制**：512.0 MB

3.2.1 题目描述

小杨同学有一串字符，里面可能有：

- 大写字母，比如 `A`、`B`、`C`
- 小写字母，比如 `a`、`b`、`c`
- 数字，比如 `0`、`1`、`2`

现在小杨同学想把这串字符变一变，规则如下：

1. 如果是大写字母，就变成对应的小写字母；
2. 如果是小写字母，就变成对应的大写字母；
3. 如果是数字，就变成 `*`。

请你按照这个规则，帮小杨把整串字符转换好。

3.2.2 输入格式

输入一共有 2 行。
第一行：一个整数，表示这串字符一共有多少个字符。
第二行：一串连续的字符，中间没有空格。

3.2.3 输出格式

输出转换后的字符。

注意：
输出时字符之间不要加空格。

3.2.4 样例

3.2.5 输入样例 1

```
5
aBc98
```

3.2.6 输出样例 1

```
AbC**
```

3.2.7 样例解释

原来的字符是：

```
aBc98
```

从左到右一个一个看：

1. `a` 是小写字母，所以变成 `A`；
2. `B` 是大写字母，所以变成 `b`；
3. `c` 是小写字母，所以变成 `C`；
4. `9` 是数字，所以变成 `*`；
5. `8` 是数字，所以变成 `*`。

所以最后输出：

```
AbC**
```

3.2.8 数据范围

字符个数不会超过 1000 个。

每个字符只会是大写字母、小写字母或数字。

2026 / 03 | 試卷練習

選擇題 15 題 × 2 分；判斷題 10 題 × 2 分；編程題 2 題 × 25 分。

以下保留所提供 PDF 的題目與原版頁碼。答案列與參考程式已移除，可在留白處寫思路；原題的樣例輸出仍完整保留。

選擇題作答

題號	答案	題號	答案	題號	答案	題號	答案
1	_____	2	_____	3	_____	4	_____
5	_____	6	_____	7	_____	8	_____
9	_____	10	_____	11	_____	12	_____
13	_____	14	_____	15	_____		

判斷題作答

題號	答案	題號	答案	題號	答案	題號	答案
1	_____	2	_____	3	_____	4	_____
5	_____	6	_____	7	_____	8	_____
9	_____	10	_____				

編程題檢查

1. 我能用一句話說清楚算法。
2. 我已手算樣例，確認輸入、輸出和下標。
3. 我已測試最小值、邊界、重複值與特殊字元。
4. 我能說明時間複雜度，並指出至少一種常見錯法。

CCF 编程能力等级认证
Grade Examination of Software Programming

C++ 三级

2026 年 03 月

1 单选题（每题 2 分，共 30 分）

第 1 题 如果字符变量 `_1` 的值是字符 `1`,那么 `(int) _1` 的值是? （ ）。

- A. 1
- B. -1
- C. 49
- D. +1 或者 -1

第 2 题 a,b是整型变量，各自有互不相同的初始值。下列程序实现了什么效果（ ）。

```
a=a^b;
b=a^b;
a=a^b;
```

- A. a,b 的值从始至终都没有改变。
- B. a,b 的值实现了互换。
- C. a,b 的值互换了以后，又还回去了，相当于没有变化。
- D. a,b 的值最后和原值不一样，没有任何意义。

第 3 题 关于下列正确的程序段，说法正确的是（ ）。

```
char str1[] = "Hello";
char str2[] ={'H','e','l','l','0'};
```

- A. 字符数组 `str1` 和 `str2` 完全相同。
- B.

```
cout<<str1<<endl;
cout<<str2<<endl;
```

这段程序多次执行将输出不同的结果。

- C. 字符数组 `str1` 和 `str2` 不相等。
- D. 这两个赋值方式完全相同。

第 4 题 关于以下程序段,说法正确的是（ ）。

```cpp
int x=10;
cout<<(x++) + (++x)<<endl;
```

- [ ] **A.** C++11 标准中，这是未定义行为，不同的环境有可能出现不同的结果
- [ ] **B.** 22
- [ ] **C.** 21
- [ ] **D.** 20

第 5 题 8 位二进制下，十进制数-15 的补码是（ ）。

- [ ] **A.** 11110000
- [ ] **B.** 10001111
- [ ] **C.** 10010000
- [ ] **D.** 11110001

第 6 题 三进制数 $2102_{(3)}$ 转换成十进制是：（ ）。

- [ ] **A.** 63
- [ ] **B.** 65
- [ ] **C.** 67
- [ ] **D.** 69

第 7 题 二进制数 10110101 是某数的 8 位补码，该数的十进制是（ ）。

- [ ] **A.** -73
- [ ] **B.** -75
- [ ] **C.** -77
- [ ] **D.** 75

第 8 题 已知 `unsigned char c = 0x0F;`（十六进制 `0F` = 二进制 `00001111`），执行 `c = c << 3;` 后，`c` 的十进制值是：（ ）。

- [ ] **A.** 64
- [ ] **B.** 72
- [ ] **C.** 80
- [ ] **D.** 120

第 9 题 补码的情况下，关于按位取反运算，用笔计算的情况下，以下说法错误的是：（ ）

- [ ] **A.** `~5` 的结果是 -6（`int` 类型，32 位）
- [ ] **B.** `~0` 的结果是 0（`int` 类型，32 位）
- [ ] **C.** `~(-3)` 的结果是 2（`int` 类型，32 位）
- [ ] **D.** `~8` 的结果是 -9（`int` 类型，32 位）

第 10 题 执行以下 C++ 代码后，`sub` 的值是（ ）。

```cpp
string str = "GESP2026";
string sub = str.substr(4, 2);
```

- [ ] **A.** 20
- [ ] **B.** 02
- [ ] **C.** 2026
- [ ] **D.** 026

第 11 题 执行以下代码后，输出结果是：（ ）。

```
int arr[] = {5, 10, 15, 20, 25, 30};
int count = sizeof(arr) / sizeof(arr[0]);
cout << count;
```

- [ ] **A.** 4
- [ ] **B.** 6
- [ ] **C.** 24
- [ ] **D.** 30

第 12 题 执行以下代码后，输出结果是：（ ）。

```
char s[10] = "abcde";
int a = sizeof(s) / sizeof(s[0]);
int b = strlen(s);
cout << a - b;
```

- [ ] **A.** 4
- [ ] **B.** 5
- [ ] **C.** 6
- [ ] **D.** 10

第 13 题 以下问题中，最不适合用枚举法解决的是：（ ）

- [ ] **A.** 找出 1~100 之间所有能被 7 整除的数
- [ ] **B.** 找出 100~200 之间的所有质数
- [ ] **C.** 计算 1+2+3+…+1000 的和
- [ ] **D.** 找出三位数中个位、十位、百位数字之和等于 10 的数

第 14 题 用枚举法解决“鸡兔同笼问题：头共 35 个，脚共 94 只，求鸡和兔的数量”，以下枚举逻辑最合理的是：（ ）

- [ ] **A.** 枚举鸡的数量 `x`（0~35），兔的数量 `y=35-x`，判断 `2*x + 4*y == 94`
- [ ] **B.** 枚举兔的数量 `y`（0~94），鸡的数量 `x=35-y`，判断 `2*x + 4*y == 94`
- [ ] **C.** 枚举所有整数 `x`（0-100）和 `y`（0-100），判断 `x+y==35 && 2*x+4*y==94`
- [ ] **D.** 枚举脚的总数 `sum`（0~94），判断 `sum == 94`

第 15 题 模拟“字符串加密”：规则为“每个字符 ASCII 码 + 3，若超过 `z` (122) 则从 `a` 重新开始”，以下代码中正确的条件判断是：（ ）

```
for (int i = 0; i < str.length(); i++) {
    // 需补充条件:
    ______________________
    else {
        str[i] += 3;
    }
}
```

- [ ] **A.** `if (str[i]> 122) str[i] = str[i] + 3 - 26;`
- [ ] **B.** `if (str[i] > 122) str[i] = str[i] - 26;`
- [ ] **C.** `if (str[i] + 3 > 122) str[i] = str[i] - 26;`
- [ ] **D.** `if (str[i] + 3 > 'z') str[i] = 'a' + (str[i] + 3 - 'z') - 1;`

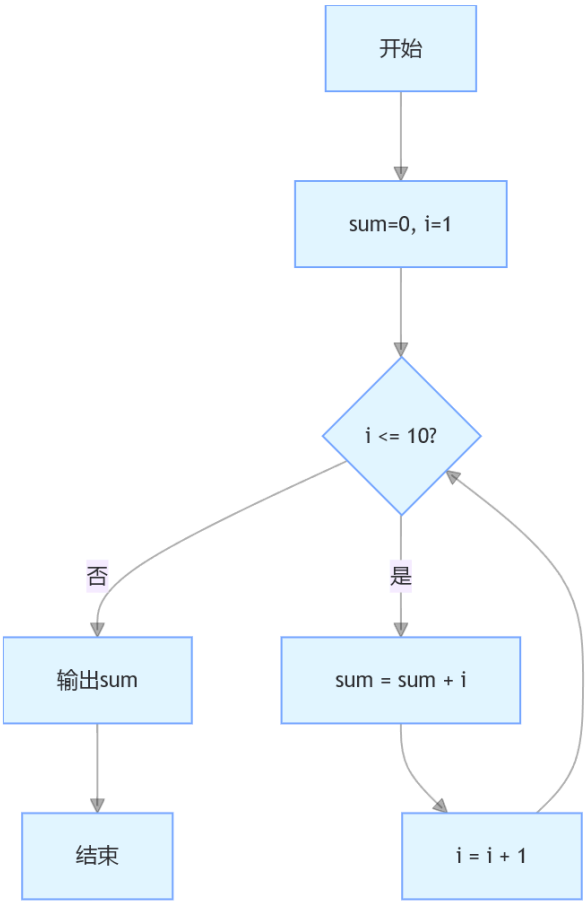
2　判断题（每题 2 分，共 20 分）

第 1 题 定义 `int arr[5] = {1,2,3};`，则 `arr[3]` 的值为 0，`arr[5]` 是合法下标。

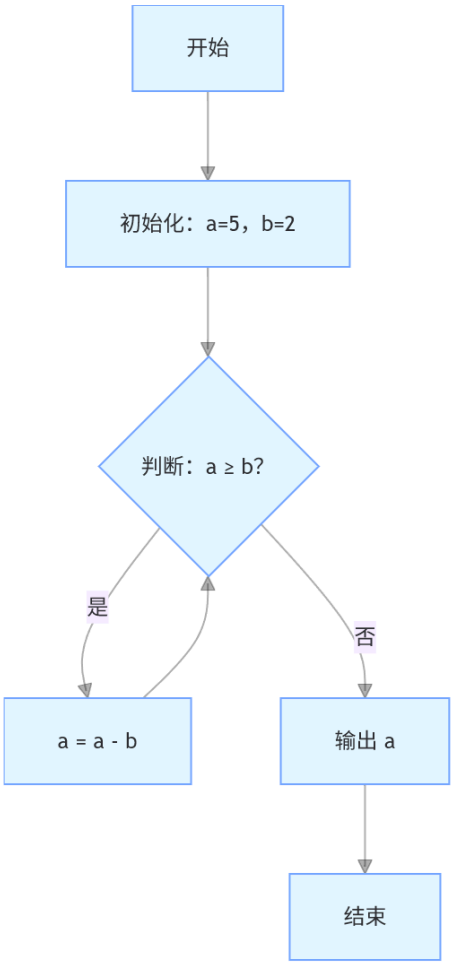
第 2 题 定义 `double arr[10];`，未手动初始化时，数组中所有元素的默认值为 0.0。

第 3 题 定义 `int arr[] = {1,2,3};`，则 `sizeof(arr)` 的结果为 12（`int` 占 4 字节）。

第 4 题 下面的流程图是用来求 1+2+3+…+10 的和。
请判断：这个流程图的逻辑正确还是错误?



第 5 题 下面流程图的功能是计算 5 对 2 取余数，输出结果为 1。



第 6 题 已知大写字符 `A` 的ASCII编码的十六进制表示为 `0x41` ，计算字符 `m` 的ASCII编码的八进制表示为 `155`（八进制）。

第 7 题 在 C++ 位运算中，各种不同的运算符有优先级的区分，使用括号能够解决优先级的问题。

第 8 题 由于在 0~255 范围内， `char` 类型和 `int` 类型可以互换，因此在这里 `x` 和 `y` 相等。

```
char x='1';
int y=1;
```

第 9 题 在C++语言中，表达式 `((0xf0 + 0x15) == 255)` 的值为 `true` 。

第 10 题 如果 `a` 为 `int` 类型的变量，且 `a` 的二进制最低位为 `0` ，则表达式 `((a & 3 & 1) == 0)` 的值为 `true` 。

3 编程题（每题 25 分，共 50 分）

3.1 编程题 1

- **试题名称**：二进制回文串
- **时间限制**：1.0 s
- **内存限制**：512.0 MB

3.1.1 题目描述

对于一个正整数 n，我们将其转换为不含前导零的二进制表示,如果这个二进制序列从左向右读与从右向左读完全相同，则称该数为二进制回文数。例如，9 的二进制表示为 $(1001)_2$，是二进制回文数；12 的二进制表示为 $(1100)_2$，不是二进制回文数。

你的任务是：给定一个正整数 n，计算在 1 到 n 的范围内二进制回文数的数量。

3.1.2 输入格式

输入一行，包含一个正整数 n。

3.1.3 输出格式

输出一行，包含一个数，表示在 1 到 n 的范围内二进制回文数的数量。

3.1.4 样例

3.1.4.1 输入样例

```
15
```

3.1.4.2 输出样例

```
6
```

3.1.5 样例解释

样例 1 中，1 到 15 范围内 1、3、5、7、9、15 是二进制回文数。

3.1.6 数据范围

$1 \leq n \leq 10^5$。

3.2 编程题 2

- **试题名称**：凯撒密码
- **时间限制**：1.0 s
- **内存限制**：512.0 MB

3.2.1 题目描述

凯撒密码是一种替换加密技术，明文中的所有字母都在字母表上向后（或向前）按照一个固定数目进行偏移后被替换成密文。例如，当偏移量是 3 的时候，所有的字母 A 将被替换成 D，B 被替换成 E，C 被替换成 F，以此类推，W 被替换成 Z，X 被替换成 A，Y 被替换成 B，Z 被替换成 C。这个加密方法是以罗马共和时期凯撒的名字命名的，据称当年凯撒曾用此方法与其将军们进行联系。

但是和所有的利用字母表进行替换的加密技术一样，凯撒密码非常容易被破解，而且在实际应用中也无法保证通信安全。

现在给你一个已破解的凯撒密码明文与密文，与一个有相同偏移量的未破解凯撒密码密文，请你帮忙破解它。

3.2.2 输入格式

输入共三行：

第一行包含一个字符串，表示已破解的凯撒密码明文；

第二行包含一个字符串，表示已破解的凯撒密码密文；

第三行包含一个字符串，表示待破解的凯撒密码密文。

3.2.3 输出格式

输出一行，包含一个字符串，表示待破解的凯撒密码对应的明文。

3.2.4 样例

3.2.4.1 输入样例

```
ABCDEFGVWXYZ
DEFGHIJYZABC
WKHTXLFNEURZQIRAMXPSVRYHUWKHODCBGRJ
```

3.2.4.2 输出样例

```
THEQUICKBROWNFOXJUMPSOVERTHELAZYDOG
```

3.2.5 样例解释

样例 1 中，通过已破解的密码得出偏移量为 `'D' - 'A' = 3`，因此，对未破解部分进行逆向偏移：密文中的 `W` 对应明文中的 `T`（`'W' - 3 = 'T'`），密文中的 `K` 对应明文中的 `H`（`'K' - 3 = 'H'`），以此类推。

3.2.6 数据范围

保证密码长度均不超过 1000，所有字符串由大写字母组成。